

Sprint #0 Report

Instructions

Objectives

- Make decisions on the Solitaire software development project.
- Learn unit testing and GUI programming in the language of your choice.

Deliverables and Grading Policy

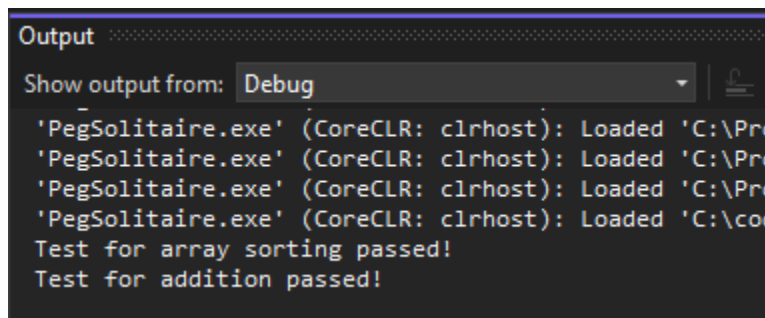
Read the “CS 449 Homework Overview” document **carefully** and make the key decisions for the software development. Use the following template to complete your report.

1. Key Decisions of the Solitaire Project (2 points)

Topic	Decision
Object-oriented programming language (if not using OOP, then justify the use of the language)	c#
GUI library (strongly encouraged)	winforms
IDE (Integrated Development Environment)	Visual studio
xUnit framework (e.g., JUnit for Java)	xunit
Programming style guide (must read it carefully)	.NET Coding Conventions - C# Microsoft Learn
Project hosting site	Github.com
Other decisions if applicable	null

2. Unit testing (4 points)

Find a tutorial on the unit test framework you have chosen and write at least two xUnit tests of a program you have written or found elsewhere. Attach here (1) the screenshot of your program execution and (2) the source code of your program.



```
Output
Show output from: Debug
'PegSolitaire.exe' (CoreCLR: clrhost): Loaded 'C:\Pro
'PegSolitaire.exe' (CoreCLR: clrhost): Loaded 'C:\Pro
'PegSolitaire.exe' (CoreCLR: clrhost): Loaded 'C:\Pro
'PegSolitaire.exe' (CoreCLR: clrhost): Loaded 'C:\cod
Test for array sorting passed!
Test for addition passed!
```

Source code:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }
}
```

```

    }

    public static int Add(int x, int y)
    {
        return x + y;
    }

    private void button1_Click(object sender, EventArgs e)
    {
        SortTests s = new SortTests();
        s.SortAnArray();
        s.Add_TwoNumbers_ReturnsSum();
        //label1.Text = s.SortAnArray();
    }

    public static T[] merge<T>(T[] arr1, T[] arr2) where T : IComparable
    {
        if (arr1.Length == 0 && arr2.Length == 1) return arr2;
        else if (arr2.Length == 0 && arr1.Length == 1) return arr1;

        if (arr1.Length == 0 && arr2.Length == 0) return arr1;

        int idx1 = 0;
        int idx2 = 0;

        T[] newArr = new T[arr1.Length + arr2.Length];
        int newArrIdx = 0;

        while (idx1 < arr1.Length && idx2 < arr2.Length)
        {
            int compareResult = arr1[idx1].CompareTo(arr2[idx2]);

            if (compareResult == -1)
            { //if evaluated arr1 value is less than evaluated arr2 val,
              then add it to the "merged" array, so it's an ascending list of values
                newArr[newArrIdx++] = arr1[idx1++];
            }
            else
            {
                newArr[newArrIdx++] = arr2[idx2++];
            }
        }

        if (idx1 == arr1.Length)

```

```

        { //one array ran out of things to add, add the rest of other arr
to newArr
            Array.Copy(arr2, idx2, newArr, newArrIdx, arr2.Length - idx2);
        }
        else
        {
            Array.Copy(arr1, idx1, newArr, newArrIdx, arr1.Length - idx1);
        }

        return newArr;
    }

    public static T[] MergeSort<T>(T[] arr) where T : IComparable
    {

        T[] left = new T[arr.Length / 2];
        T[] right = new T[arr.Length - arr.Length / 2];

        Array.Copy(arr, 0, left, 0, arr.Length / 2);
        Array.Copy(arr, arr.Length / 2, right, 0, arr.Length - arr.Length /
2);

        if (left.Length == 1 || right.Length == 1)
        {
            return merge(left, right);
        }
        else
        {
            return merge(MergeSort(left), MergeSort(right));
        }

    }

}

public class SortTests
{
    [Fact]
    public void Add_TwoNumbers_ReturnsSum()
    {
        int result = Form1.Add(2, 3);

        Assert.Equal(5, result);
        Debug.WriteLine("Test for addition passed!");
    }
}

```

```

    }

    [Fact]
    public void SortAnArray()
    {
        int[] result = Form1.MergeSort(new int[] { 5, 1, 3, 77, 3, 6});

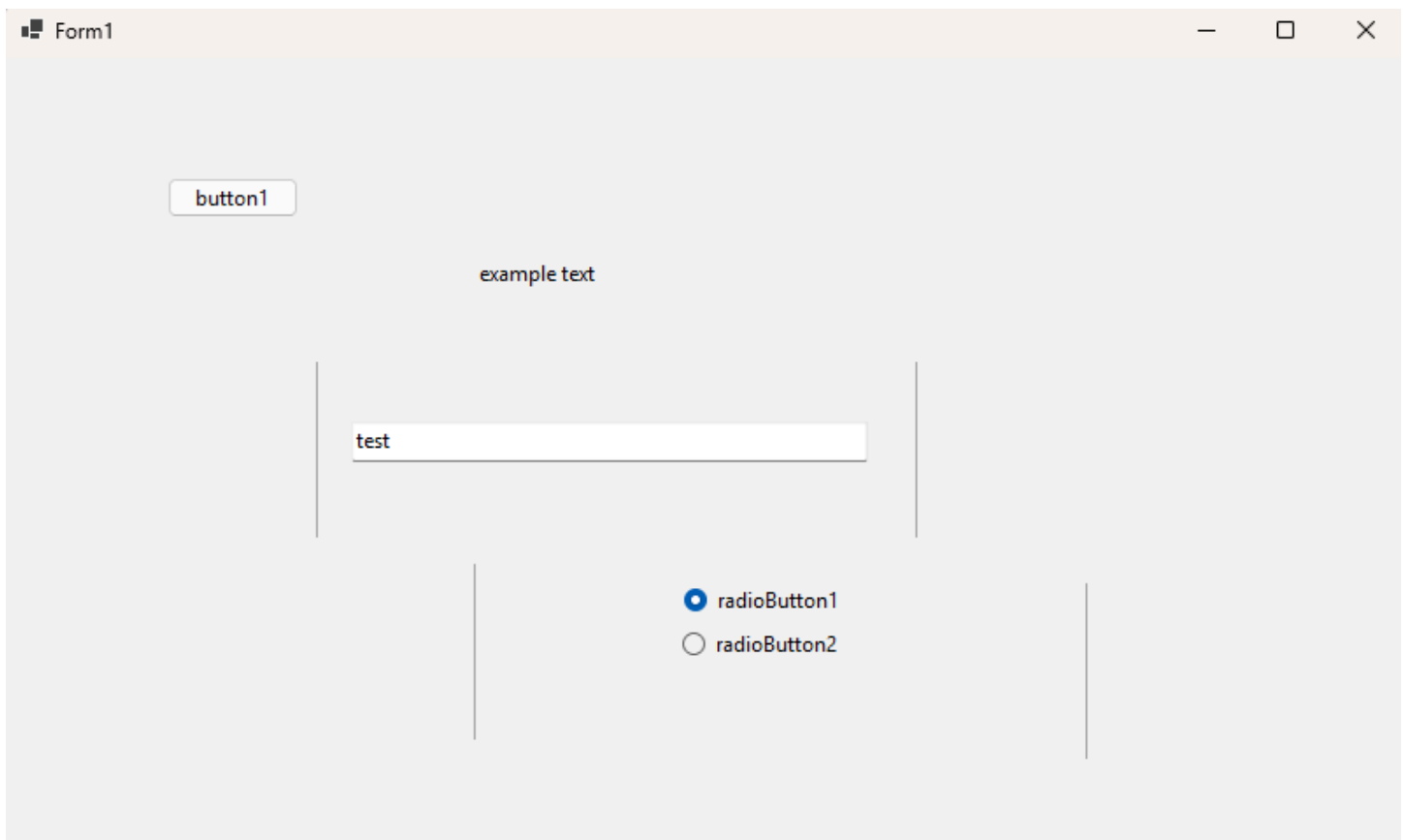
        Assert.Equal(new int[] { 1, 3, 3, 5, 6, 77}, result);
        Debug.WriteLine("Test for array sorting passed!");
    }
}

```

3. GUI programming (4 points)

Write a GUI program in the language you have chosen for your Solitaire project. The GUI of your program must include text, lines, a check box, and radio buttons. In this assignment, any GUI program of your own work is acceptable.

Attach here (1) the screenshot of your program execution and (2) the source code of your program.



Source code:

```

namespace PegSolitaire
{
    partial class Form1

```

```

{
    /// <summary>
    /// Required designer variable.
    /// </summary>
    private System.ComponentModel.IContainer components = null;

    /// <summary>
    /// Clean up any resources being used.
    /// </summary>
    /// <param name="disposing">true if managed resources should be disposed;
otherwise, false.</param>
    protected override void Dispose(bool disposing)
    {
        if (disposing && (components != null))
        {
            components.Dispose();
        }
        base.Dispose(disposing);
    }

    #region Windows Form Designer generated code

    /// <summary>
    /// Required method for Designer support - do not modify
    /// the contents of this method with the code editor.
    /// </summary>
    private void InitializeComponent()
    {
        button1 = new Button();
        textBox1 = new TextBox();
        radioButton1 = new RadioButton();
        radioButton2 = new RadioButton();
        panel1 = new Panel();
        panel2 = new Panel();
        panel3 = new Panel();
        panel4 = new Panel();
        label1 = new Label();
        SuspendLayout();
        //
        // button1
        //
        button1.Location = new Point(93, 68);
        button1.Name = "button1";
        button1.Size = new Size(75, 23);

```

```
button1.TabIndex = 0;
button1.Text = "button1";
button1.UseVisualStyleBackColor = true;
//
// textBox1
//
textBox1.Location = new Point(198, 207);
textBox1.Name = "textBox1";
textBox1.Size = new Size(295, 23);
textBox1.TabIndex = 1;
textBox1.Text = "test";
textBox1.TextChanged += textBox1_TextChanged;
//
// radioButton1
//
radioButton1.AutoSize = true;
radioButton1.Location = new Point(388, 299);
radioButton1.Name = "radioButton1";
radioButton1.Size = new Size(94, 19);
radioButton1.TabIndex = 2;
radioButton1.TabStop = true;
radioButton1.Text = "radioButton1";
radioButton1.UseVisualStyleBackColor = true;
//
// radioButton2
//
radioButton2.AutoSize = true;
radioButton2.Location = new Point(387, 324);
radioButton2.Name = "radioButton2";
radioButton2.Size = new Size(94, 19);
radioButton2.TabIndex = 3;
radioButton2.TabStop = true;
radioButton2.Text = "radioButton2";
radioButton2.UseVisualStyleBackColor = true;
//
// panel1
//
panel1.BackColor = SystemColors.ControlDark;
panel1.Location = new Point(178, 173);
panel1.Name = "panel1";
panel1.Size = new Size(1, 100);
panel1.TabIndex = 4;
//
// panel2
```

```
//
panel2.BackColor = SystemColors.ControlDark;
panel2.Location = new Point(520, 173);
panel2.Name = "panel2";
panel2.Size = new Size(1, 100);
panel2.TabIndex = 5;
//
// panel3
//
panel3.BackColor = SystemColors.ControlDark;
panel3.Location = new Point(268, 288);
panel3.Name = "panel3";
panel3.Size = new Size(1, 100);
panel3.TabIndex = 5;
//
// panel4
//
panel4.BackColor = SystemColors.ControlDark;
panel4.Location = new Point(617, 299);
panel4.Name = "panel4";
panel4.Size = new Size(1, 100);
panel4.TabIndex = 5;
//
// label1
//
label1.AutoSize = true;
label1.Location = new Point(268, 115);
label1.Name = "label1";
label1.Size = new Size(73, 15);
label1.TabIndex = 6;
label1.Text = "example text";
//
// Form1
//
AutoScaleDimensions = new.SizeF(7F, 15F);
AutoScaleMode = AutoScaleMode.Font;
ClientSize = new Size(800, 450);
Controls.Add(label1);
Controls.Add(panel4);
Controls.Add(panel3);
Controls.Add(panel2);
Controls.Add(panel1);
Controls.Add(radioButton2);
Controls.Add(radioButton1);
```

```
        Controls.Add(textBox1);  
        Controls.Add(button1);  
        Name = "Form1";  
        Text = "Form1";  
        ResumeLayout(false);  
        PerformLayout();  
    }
```

```
#endregion
```

```
private Button button1;  
private TextBox textBox1;  
private RadioButton radioButton1;  
private RadioButton radioButton2;  
private Panel panel1;  
private Panel panel2;  
private Panel panel3;  
private Panel panel4;  
private Label label1;
```

```
}
```

```
}
```